

Guida alle Correzioni in Produzione

bluethink-integration-docusign

Destinatari: Team Tecnico del Cliente | Data: Aprile 2026

Questo documento descrive **3 correzioni al codice sorgente** da applicare all'ambiente di produzione per risolvere i problemi segnalati. Tutte le modifiche riguardano un unico file Java.

File da modificare: `src/main/java/it/bars/ds/service/DSServiceHandler.java`

Come trovare il codice da modificare: Per ciascuna correzione è indicata una *stringa di ricerca*. Aprire il file in un editor di testo o IDE, usare **Ctrl+F** (Trova) e cercare esattamente quella stringa. Il cursore si posizionerà direttamente sul blocco da modificare. Non sono necessari numeri di riga.

Problema Segnalato	Causa Radice	Correzione
Alcune chiamate falliscono senza errore chiaro nei log	Nessun messaggio di log quando la ricerca del dipendente restituisce null — il documento viene ignorato silenziosamente	Correzione 3 — Aggiungere log WARN quando il dipendente non viene trovato
3 note spese Bluethink bloccate in pending (Simone e un collega)	Bug nell'estrazione del codice fiscale dal nome del file — veniva usato il separatore sbagliato	Correzione 2 — Correggere il metodo <code>getUserTaxCodeBy()</code>
Il processo Modelway non è mai partito	Cartella mancante causa <code>NullPointerException</code> , catturata silenziosamente, che blocca tutta la società	Correzione 1 — Aggiungere controllo null in <code>getFilePaths()</code>

1 Correzione 1 — Crash Silenzioso sulle Cartelle `getFilePaths`

Perché si verificava: Il metodo Java `File.listFiles()` restituisce null quando una cartella non esiste. La riga successiva chiamava `Stream.of(null)` che generava immediatamente una `NullPointerException`. Questo errore veniva catturato silenziosamente dal gestore esterno, impedendo a Modelway di elaborare qualsiasi documento, senza alcun messaggio di errore chiaro nei log.

Come trovare il codice:

Aprire `DSServiceHandler.java` → premere **Ctrl+F** → cercare:

```
Stream.of(new File(dirPath).listFiles())
```

Si troveranno **due metodi consecutivi** entrambi con questo pattern (uno senza filtro, uno con parametro `filter`) —

devono essere sostituiti **entrambi**. I metodi si chiamano `getFilePaths(String dirPath)` e `getFilePaths(String dirPath, String filter)`.

 **TROVARE questo codice (cercare: `Stream.of(new File(dirPath).listFiles())`)**

```
private Set<String> getFilePaths(String dirPath) {
    return Stream.of(new File(dirPath).listFiles())
        .filter(file -> !file.isDirectory())
        .map(File::getName)
        .collect(Collectors.toSet());
}

private Set<String> getFilePaths(String dirPath, String filter) {
    return Stream.of(new File(dirPath).listFiles())
        .filter(file -> !file.isDirectory() && file.getName().indexOf(filter) != -1)
        .map(File::getName)
        .collect(Collectors.toSet());
}
```

 **SOSTITUIRE CON (entrambi i metodi)**

```
private Set<String> getFilePaths(String dirPath) {
    File[] files = new File(dirPath).listFiles();
    if (files == null) {
        log.warn("Directory does not exist or is not accessible: " + dirPath);
        return new HashSet<>();
    }
    return Stream.of(files)
        .filter(file -> !file.isDirectory())
        .map(File::getName)
        .collect(Collectors.toSet());
}

private Set<String> getFilePaths(String dirPath, String filter) {
    File[] files = new File(dirPath).listFiles();
    if (files == null) {
        log.warn("Directory does not exist or is not accessible: " + dirPath);
        return new HashSet<>();
    }
    return Stream.of(files)
        .filter(file -> !file.isDirectory() && file.getName().indexOf(filter) != -1)
        .map(File::getName)
        .collect(Collectors.toSet());
}
```

Risultato atteso dopo la correzione: Se una cartella di Modelway non esiste sul server di produzione, il log mostrerà un messaggio WARN chiaro e l'elaborazione continuerà normalmente per le altre società (Bluethink, BlueFoundation). Il tecnico dovrà semplicemente creare le cartelle mancanti sul server.

2 Correzione 2 — Estrazione Errata del Codice Fiscale

[getUserTaxCodeBy](#)

Perché si verificava: Il metodo `getUserTaxCodeBy()` divideva il nome del file usando spazi e punti come separatori. Poiché i nomi dei file usano il trattino come separatore (es. `Bluethink-202604-Trasferta-Simone-LMRSMN94T55F1040.pdf`), il codice estraeva l'intero nome del file invece del solo codice fiscale `LMRSMN94T55F1040`. La ricerca del dipendente restituiva quindi `null` e il documento veniva saltato silenziosamente.

Come trovare il codice:

Aprire `DSServiceHandler.java` → premere **Ctrl+F** → cercare:

```
String regex = "[\\s\\.]"
```

Oppure, in alternativa, cercare il nome del metodo: `private String getUserTaxCodeBy`

È presente **un solo metodo** con questo nome nel file.

TROVARE questo codice (cercare: `String regex = "[\\s\\.]"`)

```
private String getUserTaxCodeBy(String docName) {
    String taxCode = null;

    String regex = "[\\s\\.]"
    String[] docItems = docName.split(regex);
    int docItemsSize = docItems.length;
    if (docItemsSize > 1) {
        taxCode = docItems[docItemsSize-2];
    }

    return taxCode;
}
```

SOSTITUIRE CON

```
private String getUserTaxCodeBy(String docName) {
    String taxCode = null;

    // Rimuovere prima l'estensione del file, poi dividere per il separatore dei campi
    String nameWithoutExt = docName.contains(".") ? docName.substring(0, docName.lastIndexOf('.')) :
docName;
    String[] docItems = nameWithoutExt.split(DSConstants.WORD_SEPARATOR);
    int docItemsSize = docItems.length;
    if (docItemsSize > 0) {
        taxCode = docItems[docItemsSize - 1].trim();
    }

    return taxCode;
}
```

Risultato atteso dopo la correzione: Il codice fiscale viene correttamente estratto come ultimo campo separato da trattino prima dell'estensione. Le 3 note spese bloccate di Simone e del collega verranno elaborate correttamente alla prossima esecuzione.

3 Correzione 3 — Nessun Log Quando il Dipendente Non Viene Trovato

Perché si verificava: Quando la ricerca del dipendente restituiva null (a causa del bug della Correzione 2), il documento veniva saltato senza alcun messaggio nei log. Questo rendeva impossibile capire quali documenti fossero interessati e perché.

Questa correzione deve essere applicata in **due punti distinti** all'interno del file `DSServiceHandler.java`.

Parte A — All'interno del metodo `sendEnvelopes()`

Come trovare il punto esatto — Parte A:

Aprire `DSServiceHandler.java` → premere **Ctrl+F** → cercare il nome del metodo:

```
private void sendEnvelopes
```

Una volta posizionati all'inizio del metodo `sendEnvelopes()`, scorrere verso il basso fino a trovare la riga:

```
if (companyEmployee != null) {
```

→ Questa è la **prima occorrenza** di quel blocco if nel file, ed è quella da modificare per la Parte A. La riga immediatamente sopra conterrà `companyEmployees.get(taxCode)` e la variabile `originalDocument` (non `workingDocument`) — questo vi conferma di essere nel metodo giusto.

TROVARE questo codice — Prima occorrenza (nel metodo `sendEnvelopes`, variabile: `originalDocument`)

```
DSCompanyEmployee companyEmployee = (DSCompanyEmployee)companyEmployees.get(taxCode);
if (companyEmployee != null) {
    log.debug("Company employee " + companyEmployee + " for document " + originalDocument + " to be signed");
    sendArgs = new Object[]{companyEmployee, originalDocument};
```

SOSTITUIRE CON

```
DSCompanyEmployee companyEmployee = (DSCompanyEmployee)companyEmployees.get(taxCode);
if (companyEmployee == null) {
    log.warn("No employee found for tax code [" + taxCode + "] extracted from document [" + originalDocument + "] - document skipped");
} else {
    log.debug("Company employee " + companyEmployee + " for document " + originalDocument + " to be signed");
    sendArgs = new Object[]{companyEmployee, originalDocument};
```

```
signed");
    sendArgs = new Object[]{companyEmployee, originalDocument};
```

⚠ Attenzione: Dopo questa sostituzione, la parentesi graffa chiusa `}` che terminava il blocco `if (companyEmployee != null)` ora chiude il blocco `else`. Non aggiungere né rimuovere altre parentesi graffe in quell'area.

Parte B — All'interno del metodo `retrieveEnvelopes()` `retrieveEnvelopes`

Come trovare il punto esatto — Parte B:

Aprire `DSServiceHandler.java` → premere **Ctrl+F** → cercare:

```
private void retrieveEnvelopes
```

Una volta posizionati all'inizio del metodo `retrieveEnvelopes()`, scorrere verso il basso fino a trovare:

```
if (companyEmployee != null) {
```

→ Questa è la **seconda occorrenza** nel file. Si riconosce perché la riga del log usa `workingDocument` (non `originalDocument`) — questo vi conferma di essere nel metodo giusto.

TROVARE questo codice — Seconda occorrenza (nel metodo `retrieveEnvelopes`, variabile: `workingDocument`)

```
DSCompanyEmployee companyEmployee = (DSCompanyEmployee)companyEmployees.get(taxCode);
if (companyEmployee != null) {
    log.debug("Company employee " + companyEmployee + " for document " + workingDocument + " to be
signed");
    retrieveArgs = new Object[]{companyEmployee, workingDocument};
```

SOSTITUIRE CON

```
DSCompanyEmployee companyEmployee = (DSCompanyEmployee)companyEmployees.get(taxCode);
if (companyEmployee == null) {
    log.warn("No employee found for tax code [" + taxCode + "] extracted from document [" +
        workingDocument + "] - document skipped");
} else {
    log.debug("Company employee " + companyEmployee + " for document " + workingDocument + " to be
signed");
    retrieveArgs = new Object[]{companyEmployee, workingDocument};
```

⚠ Attenzione: Stessa regola della Parte A — la parentesi graffa chiusa `}` ora chiude il blocco `else`. Non modificare altre parentesi nell'area circostante.

Risultato atteso dopo la correzione: Il log mostrerà messaggi `WARN` chiari per ogni documento saltato, indicando esattamente quale codice fiscale non è stato trovato e in quale file. Questo permette al tecnico di identificare immediatamente i dati mancanti in `users.properties`.

4 Ricompilazione e Deploy in Produzione

Dopo aver applicato tutte e 3 le correzioni, ricompilare il progetto ed eseguire il deploy sul server di produzione.

Comando di build (da eseguire nella cartella radice del progetto):

```
mvn package
```

Il file JAR prodotto si troverà in:

```
target/bt-integration-ds-1.1.jar
```

Procedura di deploy:

1. Fermare il servizio attualmente in esecuzione sul server di produzione
2. Sostituire il vecchio file JAR con quello appena compilato
3. Riavviare il servizio
4. Verificare nei log che il servizio sia partito correttamente

5 Come Verificare che le Correzioni Funzionino

Dopo il riavvio, controllare il file di log alla prima esecuzione del servizio. I messaggi attesi sono:

Messaggio nel Log	Cosa Fare
WARN – Directory does not exist or is not accessible: ../ARC_MODELWAY/...	Creare le cartelle mancanti di Modelway sul server di produzione
WARN – No employee found for tax code [XXXXYY] extracted from document [...]	Aggiungere il codice fiscale e i dati del dipendente nel file <code>users.properties</code>
INFO – Busta ... creata ed inviata per i documenti di Simone	Correzione confermata — i documenti vengono ora elaborati correttamente
Nessun WARN o ERROR per Modelway	Le cartelle esistono e il processo è partito correttamente

Posizione del file di log sul server di produzione:

```
${DS_APP}/logs/ds-app.log
```

dove `DS_APP` è la variabile d'ambiente configurata sul server. Il file viene rinnovato ogni notte a mezzanotte e conservato per 30 giorni.